



AI PROGRAMMING

编程课 机器学习与人工智能

LIFE IS SHORT, YOU NEED AI

主讲人：依力



反向传播 (Backpropagation) 算法 计算梯度

我们训练神经网络的目标是什么？

找到参数，使得损失函数最小

怎么找到让损失函数变小的参数？

先随机，然后沿梯度下降

怎么计算梯度？

$$\nabla L(w) = \left[\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right]$$

反向传播 (Backpropagation) 算法 计算梯度

神经网络是一层套一层的函数

损失函数只有一个，在最后的输出发挥作用

直观的感受是，损失函数可以计算出最后一层的梯度

那倒数第二层呢？倒数第三层呢？

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial W}$$

超参数 (Hyperparameters)

在机器学习或深度学习模型训练前设置好，并且不会通过训练自动学习得到的参数

超参数是你要人为设定的，而不是模型自己学出来的

类型	举例	是否通过训练学得?	举例含义
模型参数	权重 (weights)、偏置 (bias)	✓	模型通过反向传播学习
超参数	学习率、batch size、epoch数	✗	人为设定、影响训练过程

常见的超参数 (Hyperparameters)

类别	超参数	说明
优化相关	learning_rate (学习率)	控制每步更新幅度，太大不稳定，太小收敛慢
优化相关	optimizer (优化器)	如 SGD、Adam、RMSprop
训练策略	batch_size (批次大小)	一次训练多少样本
训练策略	num_epochs (迭代轮数)	整个数据集训练多少遍
模型结构	num_layers、hidden_size	控制网络深度与宽度
数据增强	图像裁剪、旋转、翻转等参数	提高模型泛化能力



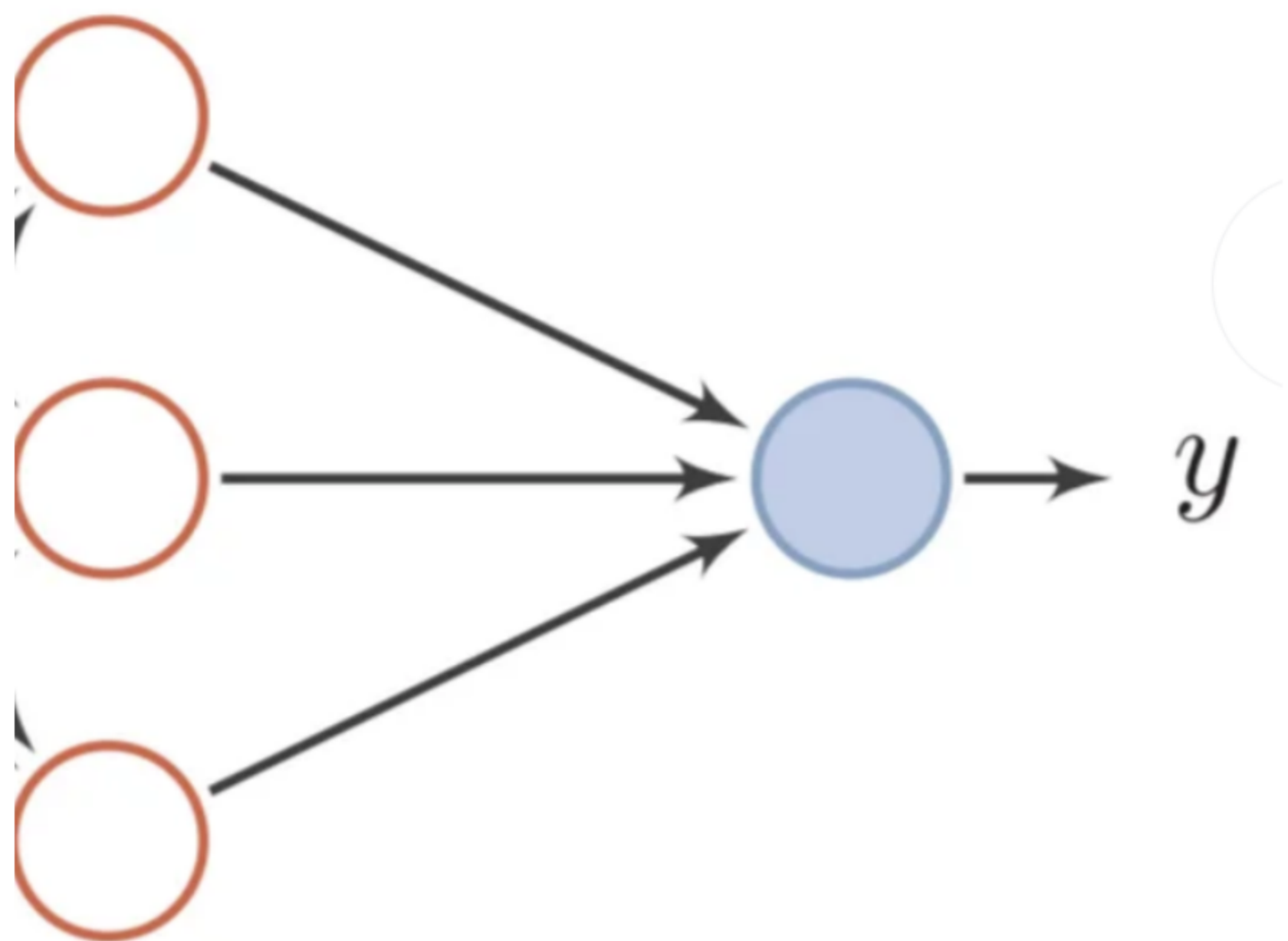
第三课

模型与算法

MACHINE LEARNING



如果标签有多个值（多个输出） 怎么办？



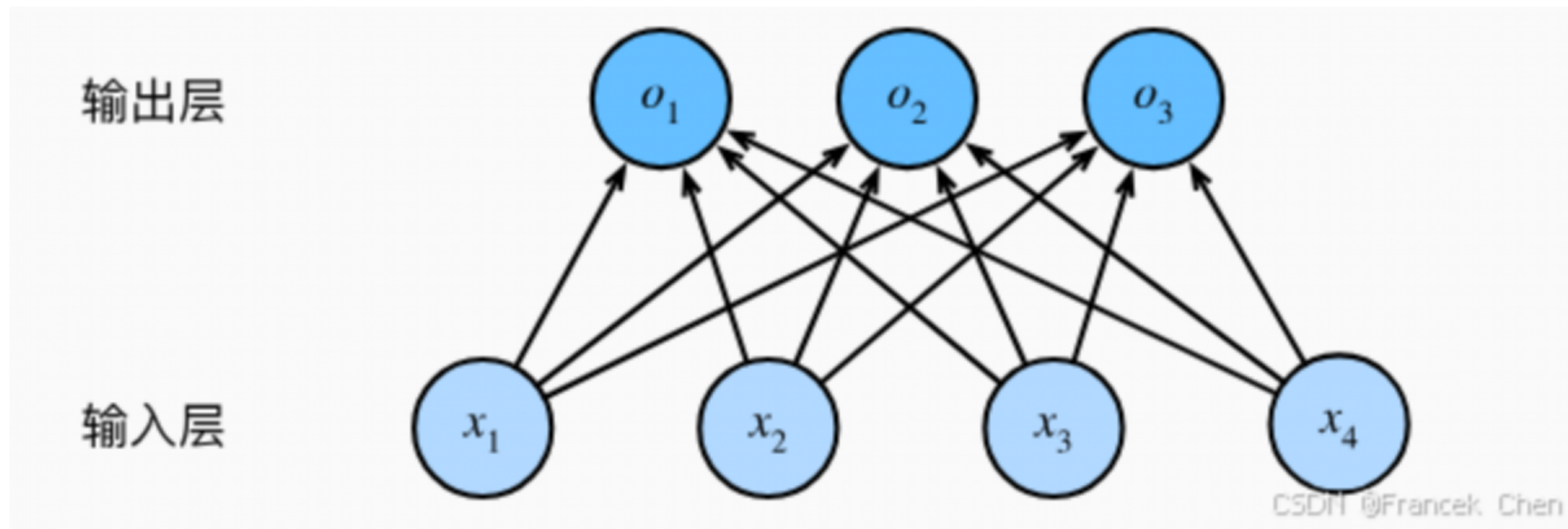
$$y_i = w_i^{\top} x + b_i$$



2° Softmax回归

多分类问题 (Multi-class Classification)

Softmax回归



给一张图片，要判断是 “T恤 🧥”、 “裤子 🩳”、 还是 “鞋子 👟”，就属于多分类问题

从线性回归拓展：Softmax回归

项目	线性回归	Softmax 回归
输出形式	连续实数	类别（概率最大对应的类别）
激活函数	无	Softmax
损失函数	均方误差（MSE）	交叉熵损失（Cross-Entropy Loss）
优化算法（目标）	最小化均方误差（MSE）	最小化交叉熵

Softmax 回归的线性打分通俗理解

鞋子  → 标签 1

衣服  → 标签 2

裤子  → 标签 3

$$z_{\text{鞋子}} = 1.2$$

$$z_{\text{衣服}} = 2.5$$

$$z_{\text{裤子}} = 0.9$$

模型觉得这个图像属于该类的可能性有多大，但这是模型的线性变换，还不代表真正的概率，不能比较和使用

Softmax 回归的数学表达

Softmax 函数 (归一化为概率)

解释:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

- $\hat{y}_j \in (0, 1)$, 代表属于类别 j 的概率;
- 所有概率加起来为 1。

$$\text{Softmax}(z) = \left[\frac{e^{2.0}}{e^{2.0} + e^{1.0} + e^{0.1}}, \frac{e^{1.0}}{e^{2.0} + e^{1.0} + e^{0.1}}, \frac{e^{0.1}}{e^{2.0} + e^{1.0} + e^{0.1}} \right] \approx [0.65, 0.24, 0.11]$$

Softmax 回归的归一化概率通俗理解

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

让所有类别的线性推导评分都变成正数

$$z_{\text{鞋子}} = 1.2 \rightarrow e^{1.2} \approx 3.32$$

$$z_{\text{衣服}} = 2.5 \rightarrow e^{2.5} \approx 12.18$$

$$z_{\text{裤子}} = 0.9 \rightarrow e^{0.9} \approx 2.46$$

Softmax 回归的归一化概率通俗理解

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}$$

让所有指数值相加作为“总值”，然后每一个类别占多少比例

$$\text{Sum} = 3.32 + 12.18 + 2.46 = 17.96$$

$$p_{\text{鞋子}} = 3.32/17.96 \approx 0.18$$

$$p_{\text{衣服}} = 12.18/17.96 \approx 0.68$$

$$p_{\text{裤子}} = 2.46/17.96 \approx 0.14$$

表示“这个图像是鞋子的概率为 18%，是衣服的概率为 68%，是裤子的概率为 14%”

Softmax 回归的归一化概率通俗理解

为什么要用Softmax? 或者说为什么会成为流行方法?

1. 将任意分数变成概率 ($0 \sim 1$, 且加起来 $= 1$)

2. 保留相对大小: 原本得分高的, 概率也大

注意打分大小指的是自己和自己比, 因为和别人比没有意义, 而且还没有到和别人比的步骤

3. 方便用于多分类的损失函数 (交叉熵) 计算

Softmax 回归的数学表达

损失函数：交叉熵损失 (Cross-Entropy Loss)

$$\mathcal{L} = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

如果真实类别是第 i 类，只有 $y_i = 1$ ，损失就变成：

$$\mathcal{L} = -\log(\hat{y}_i)$$

Softmax 回归的训练过程总结

1. 初始化参数 $\mathbf{W} \in \mathbb{R}^{K \times d}, \mathbf{b} \in \mathbb{R}^K$;
2. 对每个样本:
 - 计算 logit: $z = Wx + b$
 - 使用 softmax 得到预测概率
 - 使用交叉熵损失
3. 对损失函数反向传播, 更新参数。

Softmax 回归的交叉熵损失通俗理解

我们有模型的 预测概率，也有样本的 真实标签，我们希望衡量：

“模型预测的概率分布距离真实标签有多远？”

假设一共有C个类别

$$\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_C]$$

而真实标签是 one-hot 编码的向量

$$y = [0, 0, \dots, \frac{1}{k}, \dots, 0]$$

$$\text{Loss} = - \sum_{i=1}^C y_i \log(\hat{y}_i) = -\log(\hat{y}_k)$$

预测正确类别的概率越大，损失就越小；预测错误，损失就越大

Softmax 回归的交叉熵损失通俗理解

比如图像中既有“狗”也有“草地”：

标签： $y = [1, 0, 1, 0]$

模型预测： $\hat{y} = [0.9, 0.1, 0.8, 0.3]$

$$\text{Loss} = -[1 \cdot \log(0.9) + 0 \cdot \log(0.1) + 1 \cdot \log(0.8) + 0 \cdot \log(0.3)] = -[\log(0.9) + \log(0.8)]$$

模型只在狗和草地这两个类上承担损失惩罚

但这个时候就不能用softmax了！要用Sigmoid！

Softmax 用于
单标签分类 (Single-label classification)

Sigmoid 用于
多标签分类 (Multi-label classification)

Sigmoid激活函数

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- 输入是任意实数 $x \in (-\infty, +\infty)$
- 输出是概率 $\in (0, 1)$
- 当 x 越大, 结果越靠近 1 (表示该类更可能是正类)

Sigmoid 用于 多标签分类 (Single-label classification)

Sigmoid激活函数

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- 输入是任意实数 $x \in (-\infty, +\infty)$
- 输出是概率 $\in (0, 1)$
- 当 x 越大，结果越靠近 1（表示该类更可能是正类）

x	sigmoid(x)
-10	~0.00005
0	0.5
10	~0.99995

可以理解为“这个类别的置信度”，越接近1越像，越接近0越不像

Sigmoid 用于 多标签分类 (Single-label classification)

损失函数 二值交叉熵 Binary Cross Entropy (BCE)

$$\text{BCE} = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

$y \in \{0, 1\}$: 真实标签

$\hat{y} \in (0, 1)$: 预测概率 (来自 Sigmoid)

当真实标签是 1, 希望预测的 $\hat{y}$ 趋近于 1, 损失就会小。

当真实标签是 0, 希望预测的 $\hat{y}$ 趋近于 0, 损失也小。

预测越偏离真实值, 损失就越大。

Sigmoid 用于 多标签分类 (Single-label classification)

假设你要识别图像是否包含以下内容：

狗 🐶

草地 🌿

人 🧑

真实标签可能是：[1, 1, 0]（图中有狗和草地）

你的模型预测 logits 是：[3.0, 1.5, -2.0]

类别	真实标签 y	Logit	Sigmoid输出
狗 🐶	1	3	$\sigma(3.0) = 0.9526$
草地 🌿	1	1.5	$\sigma(1.5) = 0.8176$
人 🧑	0	-2	$\sigma(-2.0) = 0.1192$

$$\text{BCE} = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

1 对于“狗”:

- $y = 1, \hat{y} = 0.9526$
- $\text{BCE} = -[1 \cdot \log(0.9526)] = -\log(0.9526) \approx 0.0485$

2 对于“草地”:

- $y = 1, \hat{y} = 0.8176$
- $\text{BCE} = -\log(0.8176) \approx 0.2013$

3 对于“人”:

- $y = 0, \hat{y} = 0.1192$
- $\text{BCE} = -[(1 - 0) \cdot \log(1 - 0.1192)] = -\log(0.8808) \approx 0.1266$

类别	真实标签 y	Logit	Sigmoid输出	BCE 损失
狗 🐶	1	3	$\sigma(3.0) = 0.9526$	0.0485
草地 🌿	1	1.5	$\sigma(1.5) = 0.8176$	0.2013
人 🧑	0	-2	$\sigma(-2.0) = 0.1192$	0.1266

$$\text{Mean BCE} = \frac{0.0485 + 0.2013 + 0.1266}{3} \approx \boxed{0.1255}$$

模型预测得还不错（损失较小）

MNIST (Modified National Institute of Standards and Technology database)

共 7 万张 手写数字图像
(60,000 训练样本, 10,000 测试样本)

每张图片是 28×28 像素灰度图

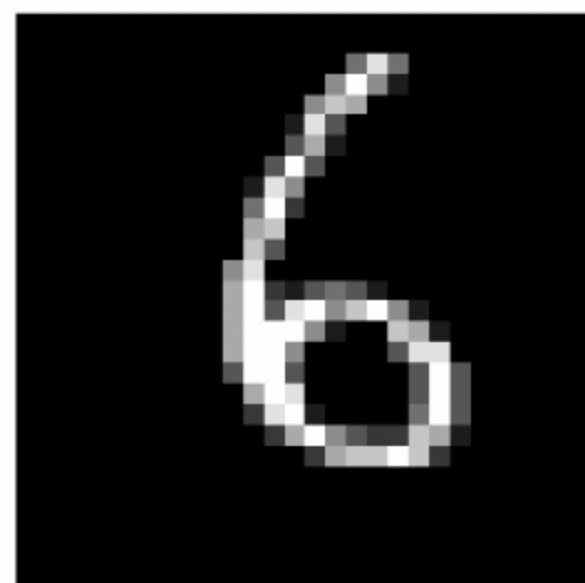
每张图片的标签是一个数字: 0-9

是许多分类算法的 入门练习数据集

Label: 3



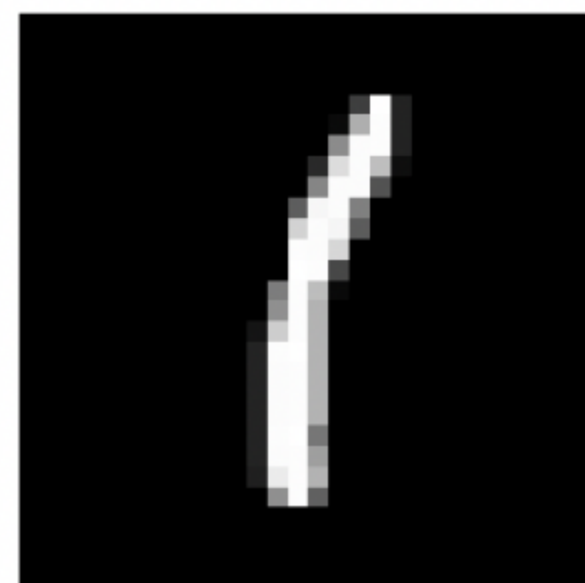
Label: 6



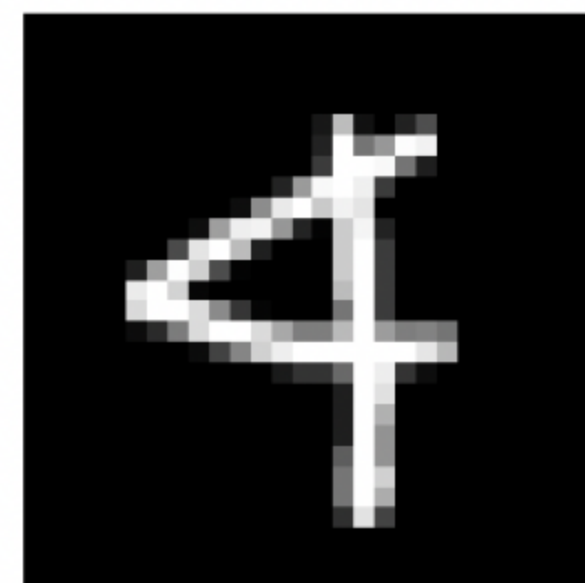
Label: 3



Label: 1



Label: 4



Label: 2



Fashion-MNIST

用于替代传统 MNIST 的一个 更加复杂、但格式相同 的数据集
也是 7 万张灰度图 (60,000 训练样本, 10,000 测试样本)

图像大小也是 28×28

每张图片是一个 服饰类别

